

Apple platform security: what the publications say, and what carries over

Prepared for David Solis, 3 Oct 2026. Read-only research. No repo changes; nothing here is a ruling on R1 to R5. Quotes are from the cited pages. "Apple's design" and "my mapping to Tetris.js" are kept apart on purpose.

Reading coverage, stated honestly. I downloaded and text-extracted the entire current Apple Platform Security PDF (266 pages, about 108,000 words) and searched all of it. I read in full only the chapters relevant to your goal: intro, operating system integrity (incl. Memory Integrity Enforcement), BlastDoor, Lockdown Mode, app security and sandboxing, the hardware formal-verification and memory-safe iBoot passages, and the revision history. I did **not** read the other chapters line by line (Apple Pay, Wallet, iCloud, HomeKit, device management, networking and others). The Private Cloud Compute Security Guide index page would not load for me; I read its "Anticipating Attacks" page and Apple's PCC blog posts, not the rest of the guide.

Corpus and what I read

Publication	Version	Read	Link
Apple Platform Security guide (formerly iOS Security; Apple calls it a "guide", not a white paper)	PDF dated August 2026 (document history lists "August 2026" newest). The web page I opened showed "March 2026", so the web copy may lag; I did not resolve this.	Whole PDF extracted; targeted chapters read	https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf and https://support.apple.com/guide/security/welcome/web
Memory Integrity Enforcement (Apple Security Research blog, 9 Sep 2025 per search metadata)	Current	Full text	https://security.apple.com/blog/memory-integrity-enforcement/
Towards the next generation of XNU memory safety: kalloc_type	Current	Fetchd (about 7,500 words); read the opening, headings and key claims, not every paragraph	https://security.apple.com/blog/towards-the-next-generation-of-xnu-memory-safety/
Private Cloud Compute blog (10 Jun 2024) and security-research post (24 Oct 2024)	Current	Design requirements read; research post read in part	https://security.apple.com/blog/private-cloud-compute/ and https://security.apple.com/blog/pcc-security-research/
PCC Security Guide, "Anticipating Attacks"	Current	Overview, section headings, limitations and conclusion	https://security.apple.com/documentation/private-cloud-compute/attacks
BlastDoor, outside analysis (Project Zero, not Apple)	Jan 2021, about iOS 14	Read earlier	https://projectzero.google/2021/01/a-look-at-imessage-in-ios-14.html
Older iOS Security editions	Archived	Not read; existence only	https://www.apple.com/mx/privacy/docs/iOS_Security_Guide_Oct_2014.pdf

I did not find a separate set of "white papers" beyond these; this is not a claim that Apple has published nothing else.

What Apple actually documents (not hypotheses)

Idea	What the source says
Security built into the design, not added	Guide intro: "Apple designs security into the core of its platforms." Key features such as hardware encryption "can't be turned off by mistake", and many are on by default so users need not configure them.
Contain the parser of untrusted input	BlastDoor "isolates, parses, transcodes, and validates untrusted data" using "sandbox restrictions and memory safe validation of output". Stated goal: "corralling attackers", increasing the effort to exploit. Known and unknown senders are segmented into distinct instances.
Small, narrow interfaces and least privilege	Apps are sandboxed with a private home directory and reach other data "only by using services explicitly provided"; privileged operations use signed entitlements; the OS partition is read-only; "APIs don't allow apps to escalate their own privileges". Option-ROM sandbox: attackers are "limited to a very narrow and well-defined sandbox interface".
Layered defenses, not one guarantee	Operating system integrity chapter lists Kernel Integrity Protection, Pointer Authentication, page protection, and Memory Integrity Enforcement (MIE) on A19 and M5 or later. MIE is secure type-aware allocators plus Enhanced Memory Tagging in synchronous mode.
Memory safety by language and by tooling	MIE post: Apple "created Swift, an easy-to-use, memory-safe language, which we employ for new code and targeted component rewrites". Guide: the iBoot bootloader's C toolchain was modified so pointers carry bounds and type information checked at run time.
Formal verification, in one narrow place	Guide: "Starting with A13 SoCs, the PKA's encryption implementations have been proved to be mathematically correct using formal verification techniques." This is the only formal-proof claim I found in my searches of the guide; it covers a hardware cryptographic block, not the OS or apps.
Enforceable, checkable guarantees (PCC)	PCC requirements: stateless computation, "enforceable guarantees" meaning components can be "constrained and analyzed", no privileged runtime access, non-targetability, verifiable transparency (researchers can verify what runs).
Attack-led evaluation	MIE: Apple's offensive research team attacked the design from 2020 to 2025 against real exploit chains; the result is described as a large reduction of exploit steps, not elimination.

Limits Apple states about itself

- MIE: "While there's no such thing as perfect security, MIE is designed to dramatically constrain attackers and their degrees of freedom during exploitation." Some issues "are able to survive MIE", for example intra-allocation buffer overflows, though Apple calls them "extremely rare".
- BlastDoor: described as a "significant obstacle", intended to "drastically improve" protection, not as a proof.
- Lockdown Mode: "makes tradeoffs for increased security at the expense of functionality, performance, or both."
- PCC: the research virtual environment cannot run every production code path; published source lacks reproducible builds, so "it is not possible to prove that the published PCC binaries are compiled from the published source code, limiting its use to that of an analysis aid." Apple also names "the risk of an unforeseen attack".
- Claim scale: MIE and BlastDoor target mercenary spyware and memory corruption. They do not address every attack class, and none of these sources claim a system is "unhackable".

My mapping to Tetris.js (hypotheses, not Apple guarantees)

Apple's documented habit	Possible JavaScript counterpart (unprototyped)
Small isolated parser, validated output	One narrow place where saved data enters, which either returns a valid state or rejects it. Matches your saved-game parsing obligation.
Narrow interfaces, least privilege	Renderer, storage and input get only read access or submit actions; the engine does not touch the browser directly.
Written threat model and stated boundaries	The v4 threat model: player in own browser, same-origin page, hostile events and timers, injection, supply chain.
Layered defenses	Proof of the core model, plus tests for what the model leaves out, plus checks at boundaries, plus CSP and deployment checks. No single layer claimed to be total.
Formal verification only where it is tractable	Prove the small core (state validity, transitions), not the whole app. Apple's one proof claim is a hardware block, which suggests narrow scope. Whether a proof tool is worth it is still your call.
Attack-led evaluation and published limits	Keep adversarial tests and mutation results as evidence, and write claims with their assumptions, e.g. "valid under assumptions A", never "unhackable".

Not carried over: hardware features (pointer authentication, memory tagging, Secure Enclave), kernel allocators, code signing and notarization. Those depend on platforms you do not control; a browser game cannot reproduce them. JavaScript is itself a managed, memory-safe-by-runtime language, so the specific memory-corruption problems in these papers differ from a JS game's risks (logic errors, invalid state, hostile input, supply chain).

Still open

- Rulings R1 to R5 are yours; nothing here approves a repair, a rewrite or a proof tool.
- Gaps to close if you want full coverage: read the remaining Platform Security chapters, the rest of the PCC Security Guide, and the full kalloc_type post; resolve the March vs August 2026 edition difference.